

Certificate

This is to certify that Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater, students of the Department of Computer Science and Engineering at Chandigarh Engineering College-CGC Landran, Mohali, have satisfactorily completed the Semester Project titled "Campus Cruiser" during the academic session 2023–2024. This work is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The project entitled Campus Cruiser is a web-based campus transportation management system developed from the ground up by the above-named students. The work described in this report is original, has been carried out under the supervision of faculty of the Department of Computer Science and Engineering, and has not been submitted elsewhere in part or in full for the award of any other degree, diploma, or certification.

The report has been reviewed by the undersigned and is hereby approved for submission to the examination committee of Chandigarh Engineering College-CGC Landran, Mohali.

Signature of Project Supervisor:

Date :-

Candidate Declaration

We, Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater, students of the B.Tech. (Computer Science and Engineering) programme at Chandigarh Engineering College-CGC Landran, Mohali, hereby solemnly declare that the Project Report entitled "Campus Cruiser" is an authentic record of our own work, carried out as a partial requirement of the Semester Project for the award of the Bachelor of Technology degree in Computer Science and Engineering.

We are aware that any act of academic dishonesty, plagiarism, or misrepresentation in this declaration or in the report may result in disciplinary action as per the rules and regulations of Chandigarh Engineering College-CGC Landran.

Akshay Anand

Roll No.: _____ Signature: _____

Aarya Ratnam Sinha

Roll No.: _____ Signature: _____

Abdul Fater

Roll No.: _____ Signature: _____

Date: _____ Place: Mohali, Punjab

**Head of Department
(Signature and Seal)**

Acknowledgement

I take this opportunity to express my sincere gratitude to the **Dr. Sukhpreet Kaur** (Director- Principal Chandigarh Engineering College, Landran & HOD CSE) for providing this opportunity to carry out the present work. I would like to express my gratitude to other faculty members of Computer Science & Engineering department of CEC, Landran for providing academic inputs, guidance & Encouragement throughout the training period. The help rendered by **Mr Amandeep Ummat & Mr Sachin** Supervisor for Project is greatly acknowledged. I would also like to acknowledge the help of my Project Partner, who helped me all along the building of this project. Finally, I express my indebtedness to all who have directly or indirectly contributed to the successful completion of my semester training

Abstract

Campus Cruiser is a full-stack, web-based campus transportation management system designed and developed to address the transportation challenges faced by the academic community of Chandigarh Engineering College-CGC Landran, Mohali. The system provides a unified digital platform for bus scheduling, advance seat booking, online payment processing, and real-time notification management — replacing the ad hoc, paper-based transportation workflow that previously operated at the campus.

The platform serves three distinct user categories: students, faculty members, and system administrators. Students and faculty register exclusively using their official @cgc.edu.in email credentials, ensuring a controlled access environment. Once authenticated, users can browse real-time bus schedules, make advance seat reservations for upcoming trips, and complete payments through the integrated Razorpay payment gateway. Faculty members enjoy an additional privilege — the ability to defer payment for trips to a later date, accommodating the irregular scheduling demands of academic staff.

Administrators are provided a comprehensive dashboard to manage bus routes and schedules, approve or cancel bookings, oversee user accounts, monitor payment transactions, and broadcast schedule change notifications to all affected users via automated email.

The technology stack was carefully selected for its performance, developer productivity, and cost-effectiveness. The backend is built on Node.js with the Express.js framework, exposing a set of RESTful APIs that follow industry best practices for security and maintainability. MongoDB, accessed through the Mongoose ODM, serves as the primary data store. Firebase Authentication handles secure identity verification, while Nodemailer provides automated SMTP-based email communication. The frontend is a React.js single-page application offering a responsive, device-agnostic user experience.

The system was developed using an Agile Scrum methodology across fifteen weekly sprints, with continuous integration and iterative refinement based on stakeholder feedback. Comprehensive testing — including unit testing, integration testing, system testing, and user acceptance testing — was conducted to validate the system's correctness, security, and performance.

Campus Cruiser successfully demonstrates how a modern, cost-effective, JavaScript-based web application can digitalise and significantly improve a previously manual institutional process. The project team — Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater — presents this report as a complete documentation of the system's design, development, and evaluation.

Table of Contents

Certificate.....	i
Candidate Declaration.....	ii
Acknowledgement.....	iii
Abstract.....	iv
Table of Contents.....	v
List of Tables.....	vi
List of Figures.....	vii
List of Abbreviations.....	viii
Chapter 1: Introduction.....	1
1.1 Brief Overview of the Project.....	1
1.2 Motivation and Problem Statement.....	3
1.3 Project Objectives.....	5
1.4 Scope of the System.....	6
1.5 Key Features.....	8
1.6 Project Modules.....	10
1.7 Hardware and Software Requirements.....	11
Chapter 2: System Analysis.....	13
2.1 Literature Review.....	13
2.2 Analysis of the Existing System.....	17
2.3 Proposed System.....	18
2.4 Feasibility Study.....	19
2.5 Project Timeline Chart.....	22
2.6 Detailed Module Description.....	23
Chapter 3: System Design.....	28

3.1	System Architecture.....	28
3.2	Use Case Diagrams.....	29
3.3	Data Flow Diagrams.....	31
3.4	Entity Relationship Diagram.....	33
3.5	Class Diagram Description.....	35
3.6	Activity Diagrams.....	36
3.7	Database Schema / Data Dictionary.....	37
Chapter 4: Software Tools and Technologies.....		43
4.1	Node.js.....	43
4.2	Express.js.....	44
4.3	React.js.....	46
4.4	MongoDB and Mongoose ODM.....	47
4.5	Firebase Authentication.....	48
4.6	Razorpay Payment Gateway.....	50
4.7	Nodemailer.....	51
4.8	Additional Libraries and Tools.....	52
Chapter 5: Implementation and Testing.....		53
5.1	Development Methodology.....	53
5.2	Backend Implementation Details.....	55
5.3	Frontend Implementation Details.....	57
5.4	User Interface Descriptions.....	58
5.5	Testing Methodology.....	61
5.6	Test Cases and Results.....	64
Chapter 6: Conclusion and Future Work.....		66
6.1	Conclusion.....	66
6.2	Limitations.....	68
6.3	Future Enhancements.....	69

References.....72

List of Tables

Table 1.1	Hardware Requirements	12
Table 1.2	Software Requirements	12
Table 2.1	Deficiencies of the Existing System	17
Table 2.2	Proposed System vs. Existing System	18
Table 2.3	Infrastructure Cost Analysis	21
Table 2.4	Project Timeline Chart	22
Table 2.5	Sprint and Team Member Mapping	23
Table 3.1	Student Use Cases	29
Table 3.2	Admin Use Cases	30
Table 3.3	Users Collection Schema	37
Table 3.4	Schedules Collection Schema	38
Table 3.5	Bookings Collection Schema	39
Table 3.6	Payments Collection Schema	40
Table 3.7	Notifications Collection Schema	41
Table 4.1	Express.js Routing Structure	45
Table 4.2	Razorpay Supported Payment Methods	50
Table 4.3	Additional Libraries and Tools	52
Table 5.1	Team Responsibilities	54
Table 5.2	UAT Results Summary	63
Table 5.3	Test Cases and Results	64

List of Figures

Figure 3.1	Three-Tier System Architecture Diagram	28
Figure 3.2	Use Case Diagram — Student and Faculty Actors	29
Figure 3.3	Use Case Diagram — Admin Actor	30
Figure 3.4	Level 0 DFD — Context Diagram	31
Figure 3.5	Level 1 DFD — System Process Decomposition	32
Figure 3.6	Level 2 DFD — Booking and Payment Subprocess	33
Figure 3.7	Entity Relationship Diagram	34
Figure 3.8	Class Diagram	35
Figure 3.9	Activity Diagram — Student Booking Flow	36
Figure 3.10	Activity Diagram — Faculty Booking Flow	36
Figure 3.11	Activity Diagram — Admin Schedule Management	37
Figure 5.1	Home Page Screenshot	58
Figure 5.2	Registration Page Screenshot	59
Figure 5.3	Schedule Browsing Page Screenshot	59
Figure 5.4	Booking Modal Screenshot	60
Figure 5.5	Booking History Page Screenshot	60
Figure 5.6	Admin Dashboard Screenshot	61

List of Abbreviations

API	Application Programming Interface
BSON	Binary JavaScript Object Notation
CGC	Chandigarh Group of Colleges
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSE	Computer Science and Engineering
DFD	Data Flow Diagram
ERP	Enterprise Resource Planning
GUI	Graphical User Interface
HMAC	Hash-based Message Authentication Code
HOD	Head of Department
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
INR	Indian National Rupee
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
MERN	MongoDB, Express.js, React.js, Node.js
MVP	Minimum Viable Product
NEFT	National Electronic Funds Transfer
NoSQL	Not Only Structured Query Language
npm	Node Package Manager
ODM	Object Document Mapper
ORM	Object Relational Mapper
PWA	Progressive Web Application
QR	Quick Response (code)
RAM	Random Access Memory
REST	Representational State Transfer
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SIS	Student Information System

SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
SSO	Single Sign-On
TDR	Transaction Discount Rate
UAT	User Acceptance Testing
UID	Unique Identifier
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UI	User Interface
UX	User Experience

Chapter 1: Introduction

1.1 Brief Overview of the Project

Campus Cruiser is a comprehensive, web-based transportation management system conceptualised and built specifically for the academic community of Chandigarh Engineering College-CGC Landran, Mohali. Transportation within a large educational campus presents unique logistical challenges — coordinating shuttle routes, managing passenger loads, handling payments, and communicating schedule changes across hundreds of users demands a robust, scalable digital solution.

Campus Cruiser directly addresses these challenges by providing an end-to-end platform where students, faculty, and administrators can interact with the transportation infrastructure in a streamlined, paperless manner. Users register using their official institute email credentials, ensuring a controlled access environment. Once authenticated, they can browse live bus schedules, make advance bookings for trips, and make payments through secure online channels powered by the Razorpay payment gateway.

Developed as a full-stack web application by Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater, Campus Cruiser replaces the fragmented, manual transportation workflow at Chandigarh Engineering College-CGC Landran with a single, integrated platform. The system enables students and faculty to register securely using their institute-issued email credentials, browse live bus schedules, make advance seat reservations, and pay online through the Razorpay payment gateway. Simultaneously, administrators gain a powerful dashboard to manage routes, monitor bookings, communicate schedule changes, and oversee financial transactions — all from a single web interface.

The application is built on a modern JavaScript-centric technology stack: a React.js single-page application on the frontend communicates with an Express.js REST API backend on Node.js, with MongoDB as the primary data store. Firebase Authentication secures user identity, and Nodemailer handles automated email communications. This stack was selected for its performance characteristics, rich ecosystem, active community support, and minimal infrastructure cost — making it well-suited for deployment within an academic institution's resource constraints.

Campus Cruiser is designed to be intuitive, accessible, and reliable. New users can complete registration in under two minutes. Booking a seat takes four steps. Administrators require no technical background to operate the management dashboard. The system is fully responsive, ensuring a seamless experience on both desktop browsers and mobile devices — critical given that the primary user base consists of students who rely predominantly on smartphones.

1.2 Motivation and Problem Statement

The motivation for developing Campus Cruiser arises directly from the pain points experienced by the Chandigarh Engineering College-CGC Landran community in their daily interaction with the campus bus service. Prior to this project, the transportation system operated as follows: bus schedules were posted on physical notice boards and emailed in bulk at the beginning of each semester; passengers queued physically to board buses without any guarantee of a seat; fare collection was handled in cash by the conductor; and any schedule changes were communicated through informal channels, often reaching users too late to adjust their plans.

This system, while functional at a small scale, exhibits significant deficiencies as the campus population grows:

- **Information Asymmetry:** Users have no reliable, real-time access to bus schedules. A student who missed a morning notice board post might be unaware of a schedule change until they are standing at the bus stop.
- **No Advance Reservation:** The absence of a seat booking mechanism means that buses are frequently overcrowded during peak hours (morning and evening) while under-utilised at other times. Students with early examinations or faculty with important meetings have no guarantee of a seat on critical trips.
- **Inefficient Fare Collection:** Cash-based fare collection is slow, error-prone, and difficult to audit. It also poses hygiene and safety concerns — a concern heightened in post-pandemic operations.
- **Faculty Payment Inconvenience:** Faculty members, who may have irregular or unpredictable schedules, find the requirement for exact cash at boarding inconvenient. A deferred or invoice-based payment option would better suit their professional needs.

- **Lack of Administrative Oversight:** Transport coordinators have no digital tool to monitor ridership trends, identify peak demand periods, or generate reports for procurement decisions (e.g., scheduling additional buses for examination periods).
- **Poor Communication Infrastructure:** Broadcast emails for schedule changes are often missed, and there is no mechanism for targeted notification — e.g., notifying only the users who have booked a specific trip that has been cancelled.

Campus Cruiser was designed to resolve each of these pain points systematically, transforming the campus transportation experience from a source of daily frustration into a seamless, reliable service.

1.3 Project Objectives

The project team — Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater — established the following specific, measurable objectives at the outset of the project:

1. Develop a secure, access-controlled web platform exclusively accessible to Chandigarh Engineering College-CGC Landran stakeholders, enforced through Firebase Authentication with email domain restriction (@cgc.edu.in).
2. Enable students and faculty to browse and filter live bus schedules by route, destination, time, and date — with real-time seat availability display.
3. Implement a robust advance seat booking mechanism with atomic concurrency control to prevent overbooking, even under simultaneous booking requests.
4. Integrate the Razorpay payment gateway to support multiple payment methods (UPI, net banking, debit/credit cards) at a fixed fare of Rs. 50 per trip, with a faculty-specific deferred payment option.
5. Build a comprehensive administrative interface for CRUD operations on bus schedules, user account management, booking oversight, and payment reconciliation.
6. Automate email communications using Nodemailer for booking confirmations, payment receipts, schedule change notifications, and cancellation alerts.
7. Deliver a fully responsive, cross-browser user interface built in React.js that provides a consistent experience on desktop and mobile devices.

8. Achieve near-zero infrastructure cost through strategic use of free-tier cloud services (MongoDB Atlas, Firebase, Razorpay sandbox, Render/Railway for hosting).

1.4 Scope of the System

The scope of Campus Cruiser defines the boundaries of what the system does and does not include in its current version.

1.4.1 In-Scope

The following functionalities are within the scope of the current implementation:

- User registration and email-domain-based access control, enforced by Firebase Authentication.
- Role-based access with three distinct roles: Student, Faculty, and Admin.
- Bus schedule management: creation, editing, deactivation, and deletion of routes by administrators.
- Real-time seat availability tracking per schedule.
- Advance seat booking by authenticated users with concurrency-safe seat decrement.
- Online payment processing via Razorpay, including signature-verified webhook handling.
- Deferred payment option exclusively for Faculty role users.
- Automated transactional email notifications via Nodemailer SMTP.
- Admin dashboard with booking management, user management, and payment monitoring.
- Booking cancellation by users (before departure) with automatic seat count restoration.
- Responsive web interface compatible with modern desktop and mobile browsers.

1.4.2 Out-of-Scope

The following are explicitly out of scope for the current version and are identified as future enhancements:

- Native iOS or Android mobile application (current system is web-only, responsive).
- Real-time GPS-based live bus tracking on a map interface.
- Integration with Chandigarh Engineering College-CGC Landran's existing ERP, SIS, or student enrollment systems.
- Multi-campus or multi-institution deployment (current system is single-campus).
- Seat-specific selection interface (seat assignment is automatic, first-come-first-served).
- QR code-based digital ticketing and conductor verification workflow.
- Advanced analytics and predictive demand modelling.

1.5 Key Features

The following are the defining features of the Campus Cruiser platform:

1.5.1 Secure, Domain-Restricted Access

Campus Cruiser is not a public platform. Registration is limited to users possessing a valid @cgc.edu.in email address. Firebase Authentication verifies the email, sends a verification link, and only allows access after the email address is confirmed. This ensures that the platform is used exclusively by the Chandigarh Engineering College-CGC Landran community, protecting both user data and transport resources.

1.5.2 Real-Time Schedule Browsing

Users are presented with a live, filterable list of all active bus schedules. Each schedule card displays the route name, source and destination, departure and arrival times, remaining seat count, and fare. Schedules with zero available seats are visually distinguished with a 'Fully Booked' badge, and their booking buttons are disabled. Filters allow users to narrow down schedules by destination, time of day, or date.

1.5.3 Advance Seat Booking with Concurrency Safety

Users can book a seat on any schedule up to 24 hours before departure. The booking process uses MongoDB's atomic `findOneAndUpdate` operation to decrement available seats, ensuring that two simultaneous booking requests for the last available seat cannot both succeed. This prevents overbooking even under high concurrency.

1.5.4 Multi-Method Online Payment via Razorpay

Campus Cruiser integrates Razorpay to accept payments through UPI (PhonePe, Google Pay, BHIM), net banking (all major Indian banks), and debit/credit cards (Visa, Mastercard, RuPay). The fixed fare is Rs. 50 per trip. Payments are verified through Razorpay's HMAC-SHA256 webhook signature mechanism, ensuring that only genuine payment confirmations update booking statuses.

1.5.5 Faculty Deferred Payment

Faculty members have the option to book a trip without immediate payment, selecting the 'Pay Later' option. The outstanding amount is logged against their user profile. Administrators can view all deferred payment balances and initiate collection as appropriate. This feature significantly improves the convenience of the service for faculty users.

1.5.6 Automated Email Notifications

Nodemailer sends HTML-formatted emails for the following events: (1) booking confirmation — including booking reference, route, date/time, fare, and a reminder to arrive 5 minutes early; (2) payment receipt — including transaction ID and amount paid; (3) schedule change alert — sent to all users with confirmed bookings on a modified schedule; (4) cancellation confirmation — with refund timeline information.

1.5.7 Comprehensive Admin Dashboard

The admin interface, accessible at `/admin`, provides tools for: creating and managing bus schedules; viewing all bookings filterable by status, date, and route; managing user accounts

including suspension; monitoring payment transactions and deferred balances; and sending custom broadcast emails to all users or specific subsets.

1.6 Project Modules

The Campus Cruiser system is organised into six primary modules:

1.6.1 Authentication Module

Manages all user identity functions — registration, email verification, login, logout, session management, and account suspension. Uses Firebase Authentication on the frontend and Firebase Admin SDK on the backend for token verification.

1.6.2 Schedule Management Module

Allows administrators to define bus routes with details including source, destination, timings, total seat capacity, and fare. Users consume schedule data through a browsing interface with real-time availability. The module enforces business rules such as preventing deletion of schedules with active bookings.

1.6.3 Booking Module

Handles the complete booking lifecycle — from initial seat reservation through confirmation, modification, and cancellation. The module communicates with the Payment Module for online transactions and the Notification Module for confirmations.

1.6.4 Payment Module

Manages all financial transactions through Razorpay integration. Creates payment orders, renders the checkout widget, handles webhook events, verifies payment signatures, and records transaction details. Also manages the deferred payment flow for faculty users.

1.6.5 Notification Module

Handles all outbound email communications. Triggered by events in the Booking, Payment, and Schedule modules. Uses Nodemailer with HTML email templates for professional, structured communications.

1.6.6 Admin Module

Provides the administrative interface for system management. Includes role-based access control (admin-only routes), schedule CRUD, user management, booking management, and financial reporting.

1.7 Hardware and Software Requirements

1.7.1 Hardware Requirements

Table 1.1: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD equivalent, 2.0 GHz	Intel Core i5 or above, 3.0+ GHz
RAM	4 GB DDR4	8 GB DDR4 or above
Storage	20 GB SSD	50 GB NVMe SSD
Network	10 Mbps broadband	100 Mbps+ broadband
Client Device	Any device with modern browser	Desktop/Laptop with 4 GB RAM
Display	1280 × 720 resolution	1920 × 1080 Full HD
Browser	Chrome 90+ / Firefox 88+	Latest Chrome, Firefox, or Edge

1.7.2 Software Requirements

Table 1.2: Software Requirements

Category	Technology	Version	Purpose
Server Runtime	Node.js (LTS)	v18.x	JavaScript backend execution environment
Backend Framework	Express.js	v4.19.x	REST API server and middleware pipeline
Frontend Library	React.js	v18.x	Component-based SPA frontend
Database	MongoDB Atlas	v6.x	Cloud-hosted NoSQL document database
ODM Library	Mongoose	v8.2.3	Schema-based MongoDB interaction layer
Authentication	Firebase Auth	v9.x	Email-based user identity management
Payment Gateway	Razorpay SDK	v2.9.2	Online payment order and verification
Email Service	Nodemailer	v6.9.13	SMTP-based transactional email dispatch
HTTP Client	Axios	v1.6.8	Frontend API communication

Version Control	Git / GitHub	Latest	Source control and collaboration
-----------------	--------------	--------	----------------------------------

Chapter 2: System Analysis

2.1 Literature Review

2.1.1 Evolution of Campus Transportation Systems

Campus transportation has undergone a significant evolution over the past two decades. In the early 2000s, most universities — both in India and globally — relied entirely on physical notice boards and periodic broadcast emails to communicate bus schedules. These systems were one-directional: information flowed from the administration to the user, with no mechanism for feedback, booking, or personalisation.

The first generation of digital transport solutions appeared around 2008–2012, primarily in North American and European universities. Systems such as TransLoc and Translucent GPS introduced real-time GPS tracking and rider alert systems. However, these were enterprise-grade solutions with substantial licensing fees and deployment complexity, making them impractical for Indian academic institutions. Moreover, their payment integrations were designed around Western financial systems and did not support Indian payment methods such as UPI.

The second wave of innovation came between 2015–2020 with the proliferation of mobile-first solutions. Several Indian universities began developing in-house mobile apps for transport booking, often on Android only, with limited feature sets.

2.1.2 Web-Based Booking Systems in Academic Settings

Priya et al. (2019) conducted a study at a private engineering college in Tamil Nadu, implementing a PHP/MySQL-based web system for campus transport booking. Their results demonstrated a 42% reduction in at-stop waiting times and a 35% improvement in user satisfaction scores after deployment. However, the system lacked online payment integration, forcing users to pay in cash — identified as the primary remaining bottleneck.

Sharma and Gupta (2021) proposed and prototyped a campus shuttle management system for NIT Kurukshetra using the MERN stack (MongoDB, Express.js, React.js, Node.js). Their system demonstrated that JavaScript full-stack architectures are highly suited to academic transport platforms due to the rapid development cycle, unified language across tiers, and rich npm

ecosystem. Their prototype, however, did not implement role differentiation between student and faculty users, nor did it include deferred payment functionality.

Mehta and Choudhary (2022) at BITS Pilani implemented a React Native-based transport booking application for their campus shuttle service. Their work is notable for introducing push notifications for booking reminders. Their payment integration used PayU, which, while functional, lacks the webhook security model of Razorpay.

2.1.3 Payment Gateway Integration in Educational Platforms

Kulkarni et al. (2022) evaluated five payment gateways — PayU, CCAvenue, Instamojo, Cashfree, and Razorpay — for suitability in an Indian academic context. Their evaluation criteria included: UPI support, webhook reliability, API documentation quality, SDK support (Node.js and Python), and transaction success rate. Razorpay ranked highest across all five criteria, with particular praise for its developer sandbox, real-time webhook delivery, and HMAC-SHA256 signature verification — which our system implements for payment security.

2.1.4 Firebase Authentication in Institutional Platforms

Firebase Authentication has emerged as the de facto standard for identity management in student-developed web applications. Its email/password provider, combined with domain-based access restriction achievable through email validation rules, provides a security model that is both strong and easy to implement. Patel and Rao (2020) used Firebase Auth in an institutional platform, noting that its token-based stateless authentication model integrates seamlessly with REST APIs without requiring session storage on the backend.

2.1.5 Research Gap Addressed by Campus Cruiser

The review of existing literature and prior implementations reveals five critical gaps that Campus Cruiser addresses:

9. No existing Indian campus transport solution combines advance booking, Razorpay payment integration (with webhook verification), role-differentiated access (student vs. faculty), deferred payment for faculty, and a comprehensive admin dashboard in a single unified platform.

10. Most prior solutions do not enforce email-domain-based access restriction. Campus Cruiser uses Firebase Auth to restrict registration to @cgc.edu.in addresses only.
11. Automated, event-triggered email notifications using Nodemailer are absent from existing solutions, resulting in poor communication about schedule changes and booking confirmations.
12. Atomic concurrency control for seat booking — preventing overbooking under simultaneous requests — has not been explicitly addressed in prior work.
13. The combination of near-zero infrastructure cost and production-ready security features has not been demonstrated in the reviewed literature.

2.2 Analysis of the Existing System

2.2.1 Current Workflow at CGC Landran

Before Campus Cruiser, the transportation workflow at Chandigarh Engineering College-CGC Landran operated as follows: At the beginning of each semester, the transport office published a PDF schedule on the institute website and distributed printed copies to hostel common rooms. Updates to the schedule were communicated via a general institute mailing list. Passengers boarded buses on a first-come-first-served basis with no reservation; the conductor collected Rs. 50 in cash from each passenger. Faculty members occasionally settled dues with the transport office at the end of each month.

2.2.2 Deficiencies of the Existing System

Table 2.1: Deficiencies of the Existing System

Deficiency	Impact	Severity
No digital booking mechanism	Overcrowding, no seat guarantee, student frustration	High
Cash-only payment	Slow boarding, collection errors, no audit trail	High
Delayed schedule communication	Passengers miss buses or wait unnecessarily	High
No ridership data collection	Inability to plan fleet allocation or identify peak demand	Medium

No faculty payment flexibility	Faculty inconvenience, manual reconciliation burden	Medium
No cancellation mechanism	No way to free reserved seats or issue refunds	Medium
No user authentication	No accountability for transport usage	Low

2.3 Proposed System

The proposed system — Campus Cruiser — addresses each deficiency identified above with a corresponding digital solution. The following table maps each pain point to its resolution:

Table 2.2: Proposed System vs. Existing System

Existing System Pain Point	Campus Cruiser Solution
No booking mechanism	Web-based advance seat booking with real-time availability
Cash-only payment	Razorpay integration: UPI, cards, net banking at Rs. 50/trip
Delayed schedule communication	Automated email notifications via Nodemailer on any change
No ridership data	Admin dashboard with booking stats and transaction history
Faculty payment inconvenience	Deferred payment option with outstanding balance tracking
No cancellation flow	User-initiated cancellation with seat count restoration
No accountability	Firebase Authentication with institute email verification

2.4 Feasibility Study

2.4.1 Technical Feasibility

The technical feasibility of Campus Cruiser is firmly established. All core technologies — Node.js, Express.js, React.js, MongoDB, Firebase Authentication, Razorpay, and Nodemailer — are mature, well-documented, and widely deployed in production environments. The project team had prior experience with JavaScript, React, and Node.js from coursework, reducing the learning curve for framework-specific features.

The choice of MongoDB as the database is technically justified by the flexible, schema-less nature of the data (schedules, bookings, and user profiles evolve over time) and by the seamless

integration of BSON documents with JavaScript's native object model. The Mongoose ODM adds structure and validation without sacrificing MongoDB's flexibility.

Firebase Authentication's token-based model requires no server-side session storage, making the backend stateless and horizontally scalable. Razorpay's Node.js SDK and comprehensive documentation made payment integration technically straightforward. The sandbox testing environment allowed full end-to-end payment flow testing without real money, substantially reducing integration risk.

2.4.2 Economic Feasibility

One of Campus Cruiser's key advantages is its near-zero infrastructure cost. The following table details the monthly cost structure:

Table 2.3: Infrastructure Cost Analysis

Infrastructure Component	Provider	Monthly Cost (INR)	Notes
Database	MongoDB Atlas (M0 Free)	₹0	512 MB storage, sufficient for campus scale
Authentication	Firebase (Spark Plan)	₹0	Free up to 10,000 users/month
Payment Gateway	Razorpay	₹0 + 2% TDR	No monthly fee; 2% per transaction
Backend Hosting	Render (Free Tier)	₹0	Free tier with 512 MB RAM
Frontend Hosting	Vercel (Hobby Plan)	₹0	Free for single web application
Email Service	Gmail SMTP / Nodemailer	₹0	Free using institutional Gmail credentials
Domain Name (optional)	Any registrar	~₹600/year	Optional custom domain

2.4.3 Operational Feasibility

Campus Cruiser has been designed with operational simplicity as a first-class requirement. The admin interface requires no technical knowledge — adding a bus schedule is as straightforward as filling in a form; changing a schedule automatically notifies affected users without any additional administrative action. A single part-time administrator can comfortably manage the day-to-day operation of the system, estimated at less than one hour of active administration per day.

For end users (students and faculty), the system presents no learning curve beyond what is expected of any modern web application. The registration flow took an average of 2 minutes in user acceptance testing, and booking a trip took an average of 90 seconds including payment. These metrics confirm strong operational feasibility.

2.4.4 Schedule Feasibility

The project was planned and executed over a 15-week period, structured as seven sprints. The phased approach — beginning with backend scaffolding, progressing to core feature modules, and concluding with integration, testing, and documentation — ensured that each component was well-tested before integration. The timeline was adhered to, with no major phase overruns.

2.5 Project Timeline Chart

Table 2.4: Project Timeline Chart

Sprint / Phase	Duration	Key Deliverables	Team Member Lead
Sprint 1: Requirements & Design	Weeks 1–3	Requirement docs, wireframes, DB schema, API contracts	All members
Sprint 2: Backend Core	Weeks 3–6	Express API setup, MongoDB models, Firebase auth middleware	Akshay Anand
Sprint 3: Frontend Core	Weeks 5–8	React app scaffold, routing, schedule browsing, auth pages	Aarya Ratnam Sinha
Sprint 4: Booking & Payment	Weeks 7–10	Booking module, Razorpay integration, webhook handling	Abdul Fater
Sprint 5: Notifications & Admin	Weeks 10–12	Nodemailer setup, email templates, admin dashboard CRUD	Akshay Anand & Abdul Fater
Sprint 6: Integration & Testing	Weeks 12–14	End-to-end testing, bug fixes, security testing, UAT	All members
Sprint 7: Deployment & Docs	Week 15	Cloud deployment, README, final project report	Aarya Ratnam Sinha

2.6 Detailed Module Description with All Functionalities

2.6.1 User Authentication Module

The authentication module is the gateway to all other system functions. New users arrive at the /register page and provide their full name, official @cgc.edu.in email address, role (Student or Faculty), phone number, and a password of at least 8 characters. Client-side validation in React checks for email format, password strength, and completeness before the form is submitted.

On form submission, the React frontend calls the Firebase `createUserWithEmailAndPassword()` function, which creates a new Firebase user account and immediately sends a verification email to the provided address. The frontend then makes a POST request to the backend /api/auth/register endpoint, providing the Firebase UID alongside the user profile data. The backend stores this profile in the MongoDB users collection, associated with the Firebase UID for future token validation.

Login uses Firebase `signInWithEmailAndPassword()`. If the account's email has not been verified, a warning is displayed and access is denied. Upon successful login, Firebase provides an ID token (a signed JWT with a 1-hour expiry). This token is stored in the browser's localStorage and attached as a Bearer token in the Authorization header of every subsequent API request.

2.6.2 Bus Schedule Management Module

The schedule module serves as the central data source for the booking workflow. Each schedule document in MongoDB contains: routeName, source, destination, departureTime (HH:MM 24-hour format), arrivalTime, totalSeats, availableSeats (decremented on booking), fare (defaulting to Rs. 50), isActive (a boolean flag), and createdBy (the admin's user ObjectId).

Administrators access the schedule management section of the admin dashboard at /admin/schedules. The interface presents a paginated table of all schedules (active and inactive) with inline edit and delete controls. Attempting to delete a schedule that has confirmed bookings triggers a warning and offers the option to deactivate instead of delete, preserving booking integrity.

When a schedule is modified (e.g., departure time changes), the system queries the bookings collection for all confirmed bookings linked to the affected schedule and dispatches a schedule change email to each affected user via the Notification Module.

2.6.3 Booking Module

The booking module manages the complete lifecycle of a user's trip reservation. A user initiates a booking by clicking 'Book Now' on a schedule card. A confirmation modal displays the full trip details: route, source, destination, departure date and time, fare, and the user's name. The user selects the trip date from a date picker (restricted to future dates within a configurable booking window, defaulting to 24 hours to 30 days in advance).

On confirming the booking details, the frontend makes a POST request to `/api/bookings/create` with the `scheduleId`, `userId`, `tripDate`, and `paymentMode` (online or deferred). The backend validates the request, checks that the schedule is active, and then performs an atomic seat decrement using MongoDB's `findOneAndUpdate`. If the decrement succeeds, a booking document is created with status 'pending' and a unique `bookingRef` is generated. If `availableSeats` is already 0, the backend returns a 409 Conflict.

2.6.4 Payment Module

The payment module handles all financial transactions. For 'online' payment mode, the backend creates a Razorpay order by calling the Razorpay Orders API with `amount`: 5000 (representing Rs. 50 in paise) and `currency`: 'INR'. The order ID is returned to the frontend, which initialises the Razorpay checkout widget.

After the user completes payment on the Razorpay widget, Razorpay triggers a POST request to the backend's webhook endpoint (`/api/payments/webhook`). The backend validates the event by computing HMAC-SHA256 of the raw request body using the Razorpay webhook secret and comparing it to the `X-Razorpay-Signature` header. If valid, the booking status is updated to 'confirmed', the payment record is created, and the Notification Module is triggered to send a confirmation email.

For Faculty users selecting 'Pay Later', the booking is immediately confirmed (no payment processing), and the user's `outstandingBalance` field in the `users` collection is incremented by Rs. 50. Administrators can view all outstanding balances and initiate settlement through the admin panel.

2.6.5 Notification Module

The notification module is a service layer that wraps Nodemailer with pre-built HTML email templates. Nodemailer is configured with the institute's SMTP credentials (or a dedicated Gmail

account using an app-specific password). Email templates are constructed as HTML strings with inline CSS for maximum email client compatibility.

The module exposes the following functions: `sendBookingConfirmation()`, `sendPaymentReceipt()`, `sendScheduleChangeAlert()`, and `sendCancellationConfirmation()`. Each function constructs the appropriate HTML body and calls `nodemailer.sendMail()` with the formatted content.

2.6.6 Admin Module

The admin module provides a protected set of routes and a React-based dashboard for system management. Admin routes are doubly protected: first by the Firebase token verification middleware, then by a `roleCheck` middleware that queries the user's role in MongoDB and returns 403 Forbidden for non-admin roles.

The dashboard homepage displays summary metrics: today's total bookings, today's revenue, total active schedules, and total registered users. These metrics are computed from MongoDB aggregation queries and returned via the `/api/admin/stats` endpoint. Below the metrics, the dashboard provides navigation to: Schedule Management, Booking Management, User Management, and Payment Management.

Chapter 3: System Design

3.1 System Architecture

Campus Cruiser is built on a three-tier client-server architecture, which cleanly separates the presentation logic (React.js frontend), the application logic (Node.js/Express.js REST API), and the data storage layer (MongoDB). This architecture is augmented by two external service integrations: Firebase Authentication (identity management) and Razorpay (payment processing).

The architectural flow is as follows: The user interacts with the React.js frontend through a web browser. All user actions that require data are translated into HTTP REST API calls via Axios to the Express.js backend. Each API request carries a Firebase ID token in the Authorization header. The Express.js middleware layer validates the token using the Firebase Admin SDK before passing the request to the appropriate route handler. Route handlers execute business logic and interact with MongoDB through Mongoose models. For payment events, Razorpay's webhook system sends events directly to the backend's webhook endpoint.

This architecture is intentionally stateless at the API level — no session data is stored on the backend server. All authentication state lives in the Firebase token, which each request carries. This makes the backend horizontally scalable: multiple Node.js instances can serve requests without coordination, as each request is self-contained.

3.2 Use Case Diagrams

3.2.1 Actor Description

The system defines three actor types with hierarchical access levels:

- Student: Authenticated user with basic access to schedule browsing, booking, online payment, booking history, and cancellation.
- Faculty: Extends Student with the additional ability to book trips with deferred payment and view outstanding balance.

- Admin: Has all student/faculty capabilities plus full system management — schedule CRUD, user management, booking management, payment management, and notification broadcast.

3.2.2 Student and Faculty Use Cases

Table 3.1: Student and Faculty Use Cases

Use Case	Actor	Pre-condition	Post-condition
UC01: Register Account	Student/Faculty	User has @cgc.edu.in email	Account created; verification email sent
UC02: Verify Email	Student/Faculty	Account registered	Email verified; access granted
UC03: Login	Student/Faculty	Email verified, credentials valid	Firebase ID token issued; session active
UC04: Browse Schedules	Student/Faculty	Logged in	List of active schedules with availability shown
UC05: Book a Trip	Student/Faculty	Logged in; seat available	Provisional booking created; payment initiated
UC06: Make Online Payment	Student	Provisional booking exists	Razorpay checkout presented; payment processed
UC07: Defer Payment	Faculty only	Provisional booking exists	Booking confirmed; outstandingBalance incremented
UC08: View Booking History	Student/Faculty	Logged in	List of past and upcoming bookings displayed
UC09: Cancel Booking	Student/Faculty	Confirmed booking before departure	Booking cancelled; seat count restored; email sent
UC10: Logout	Student/Faculty	Logged in	Firebase token revoked; session cleared

3.2.3 Admin Use Cases

Table 3.2: Admin Use Cases

Use Case	Description
UC11: Manage Schedules	Create, edit, deactivate, and delete bus routes and timings
UC12: Manage Users	View, search, suspend, and reactivate user accounts
UC13: View All Bookings	Browse all bookings with filtering by date, status, route, user
UC14: Cancel Booking (Admin)	Forcibly cancel any booking with automatic notification to user

UC15: Monitor Payments	View completed transactions, pending deferred amounts, revenue totals
UC16: Broadcast Notification	Send a custom email to all users or a filtered subset
UC17: View Dashboard Metrics	View summary stats: bookings, revenue, active schedules, users

3.3 Data Flow Diagrams

3.3.1 Level 0 DFD — Context Diagram

At the context level, Campus Cruiser is represented as a single system process. Four external entities interact with it: (1) Student/Faculty — inputs authentication data, booking requests, and payment interactions; receives schedule information, booking confirmations, and email notifications; (2) Admin — inputs schedule data, management commands, and broadcast messages; receives system status reports and metrics; (3) Razorpay Gateway — receives payment order creation requests; sends webhook payment confirmation events; (4) Email SMTP Server — receives outbound email dispatch requests from Nodemailer; delivers emails to recipients.

3.3.2 Level 1 DFD — System Process Decomposition

At Level 1, the system decomposes into seven major processes:

14. P1 — User Authentication Process: Accepts credentials from user; validates with Firebase Admin SDK; returns session token and user profile.
15. P2 — Schedule Management Process: Receives schedule data from Admin; stores/updates in Schedule data store; returns schedule list to all users.
16. P3 — Booking Process: Receives booking request from authenticated user; checks seat availability; creates provisional booking.
17. P4 — Payment Process: Creates Razorpay order; receives webhook confirmation; verifies signature; updates booking status.
18. P5 — Notification Process: Triggered by P3 (booking confirm), P4 (payment receipt), P2 (schedule change). Sends email via SMTP.
19. P6 — Cancellation Process: Receives cancellation request; validates ownership and timing; restores seat count; triggers P5.
20. P7 — Admin Management Process: Provides CRUD operations across all data stores with role verification.

3.3.3 Level 2 DFD — Booking and Payment Subprocess

The booking and payment subprocess is the most critical flow, decomposed into the following twelve steps:

21. User sends bookingRequest {scheduleId, tripDate, userId, paymentMode}.
22. P3.1 — Validate Request: Check that schedule exists, is active, and user has no duplicate booking for same schedule and date.
23. P3.2 — Seat Availability Check: Query Schedules data store for availableSeats. If 0, return SEAT_UNAVAILABLE error.
24. P3.3 — Atomic Seat Decrement: Execute MongoDB findOneAndUpdate with \$inc:{availableSeats:-1} and condition {availableSeats:{>0}}.
25. P3.4 — Create Provisional Booking: Insert booking record with status=pending.
26. P4.1 — Create Razorpay Order: POST to Razorpay /v1/orders with amount=5000, currency=INR.
27. P4.2 — Return Order ID to Frontend: Frontend initialises Razorpay checkout widget.
28. P4.3 — User Completes Payment on Razorpay widget.
29. P4.4 — Razorpay Webhook Event: POST to /api/payments/webhook with payment.captured event.
30. P4.5 — Verify Signature: HMAC-SHA256(rawBody, webhookSecret) === X-Razorpay-Signature header.
31. P4.6 — Update Booking Status: Set booking.status = confirmed; create Payment record.
32. P5.1 — Trigger sendBookingConfirmation() and sendPaymentReceipt() via Nodemailer.

3.4 Entity Relationship Diagram

3.4.1 Entities, Attributes, and Primary Keys

Campus Cruiser's data model consists of five primary entities:

- User (_id PK, fullName, email, role, phone, firebaseUid UK, isActive, outstandingBalance, createdAt)

- Schedule (_id PK, routeName, source, destination, departureTime, arrivalTime, totalSeats, availableSeats, fare, isActive, createdBy FK→User)
- Booking (_id PK, userId FK→User, scheduleId FK→Schedule, tripDate, bookingDate, status, paymentMode, paymentId FK→Payment, bookingRef UK)
- Payment (_id PK, bookingId FK→Booking, userId FK→User, razorpayOrderId, razorpayPaymentId, amount, status, paidAt)
- Notification (_id PK, recipientEmail, subject, bodyHtml, type, sentAt, relatedBookingId FK→Booking)

3.4.2 Relationships

- A User can make many Bookings (one-to-many: User to Booking).
- A Schedule can have many Bookings (one-to-many: Schedule to Booking).
- Each Booking has exactly one Payment record (one-to-one: Booking to Payment, optional for deferred).
- Each Booking or Admin action can generate one or more Notifications (one-to-many).
- One Admin (User) can create many Schedules (one-to-many: User to Schedule).

3.5 Class Diagram Description

The Campus Cruiser backend is organised around the following class/module hierarchy:

- UserModel: Mongoose schema class encapsulating user attributes. Methods: findByFirebaseUid(), updateBalance().
- ScheduleModel: Mongoose schema for bus routes. Methods: findAvailable(destination, date), decrementSeat().
- BookingModel: Mongoose schema with population support for userId and scheduleId. Methods: cancel(), confirm().
- PaymentModel: Stores Razorpay transaction details. Static method: verifyWebhookSignature(body, signature).

- **AuthMiddleware:** Express middleware. Method: `verifyFirebaseToken(req, res, next)`. Attaches decoded user to `req.user`.
- **RoleMiddleware:** Express middleware. Method: `requireRole(role)(req, res, next)`.
- **NotificationService:** Service class with methods: `sendBookingConfirmation()`, `sendPaymentReceipt()`, `sendScheduleChangeAlert()`, `sendCancellationConfirmation()`.
- **RazorpayService:** Service class wrapping the Razorpay SDK. Methods: `createOrder(amount)`, `verifySignature(body, signature)`.

3.6 Activity Diagrams

3.6.1 Student Booking Activity

Start → User visits `/schedules` → System fetches active schedules from API → Schedules rendered in card grid → User applies optional filters → User clicks 'Book Now' → System checks authentication → [Not authenticated] → Redirect to `/login` → [Authenticated] → Booking confirmation modal displayed → User selects trip date → User reviews details → [Student role] → Razorpay checkout rendered → User completes payment → [Payment success] → Booking confirmed; email dispatched → Success screen with booking reference → [Payment failed] → Error displayed, provisional booking reversed → End.

3.6.2 Faculty Booking Activity

Start → Faculty user clicks 'Book Now' → Authentication check → Booking confirmation modal → Faculty selects trip date → [Pays Now] → Same as Student flow above → [Pay Later] → Backend confirms booking immediately → `outstandingBalance += 50` → Booking confirmation email dispatched → Success screen → End.

3.6.3 Admin Schedule Management Activity

Start → Admin logs in → Admin accesses `/admin/schedules` → System fetches all schedules → Admin selects action → [Create] → Schedule creation form → Admin fills route, timing, seats, fare → Save to MongoDB → [Edit] → Schedule edit modal → Admin modifies fields → Save → If departure time changed, system queries affected bookings → Sends change notification

emails → [Delete] → System checks for active bookings → [Has active bookings] → Deactivation offered instead → [No active bookings] → Schedule deleted → End.

3.7 Database Schema / Data Dictionary

3.7.1 Users Collection

Table 3.3: Users Collection Schema

Sr.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Primary Key, Auto	MongoDB auto-generated unique document ID
2	fullName	String	Required, max 100 chars	Full legal name of the user
3	email	String	Required, Unique, lowercase	Institute email — must end with @cgc.edu.in
4	role	String	Enum: student/faculty/admin	User role controlling access level
5	phone	String	Optional, max 15 chars	User contact number
6	firebaseUid	String	Required, Unique	Firestore Authentication UID
7	isActive	Boolean	Default: true	Account active/suspended status
8	outstandingBalance	Number	Default: 0	Cumulative unpaid fare (Faculty only)
9	createdAt	Date	Auto (timestamps)	Account creation timestamp
10	updatedAt	Date	Auto (timestamps)	Last update timestamp

3.7.2 Schedules Collection

Table 3.4: Schedules Collection Schema

Sr.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Primary Key, Auto	MongoDB auto-generated unique ID
2	routeName	String	Required, max 150 chars	Descriptive name of the route
3	source	String	Required, max 100 chars	Departure location name
4	destination	String	Required, max 100 chars	Arrival location name
5	departureTime	String	Required, format HH:MM	Scheduled departure time in 24-hour format
6	arrivalTime	String	Required, format HH:MM	Expected arrival time in 24-hour format

7	totalSeats	Number	Required, min: 1, max: 100	Total passenger capacity of the bus
8	availableSeats	Number	Required, min: 0	Current number of bookable seats
9	fare	Number	Default: 50	Fare per trip in Indian Rupees (INR)
10	isActive	Boolean	Default: true	Controls visibility in public schedule listing
11	createdBy	ObjectId	Ref: User, Required	Admin who created this schedule entry

3.7.3 Bookings Collection

Table 3.5: Bookings Collection Schema

Sr.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Primary Key, Auto	MongoDB auto-generated unique ID
2	userId	ObjectId	Ref: User, Required	User who made the booking
3	scheduleId	ObjectId	Ref: Schedule, Required	Schedule that was booked
4	tripDate	Date	Required	The actual date of travel
5	bookingDate	Date	Auto (timestamps)	Timestamp when booking was created
6	status	String	Enum: pending/confirmed/cancelled	Current state of the booking lifecycle
7	paymentMode	String	Enum: online/deferred	Payment method selected at booking time
8	paymentId	ObjectId	Ref: Payment, Optional	Link to the associated payment record
9	bookingRef	String	Required, Unique	Human-readable booking reference (e.g., CC-2024-00847)

3.7.4 Payments Collection

Table 3.6: Payments Collection Schema

Sr.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Primary Key, Auto	MongoDB auto-generated unique ID
2	bookingId	ObjectId	Ref: Booking, Required	The booking this payment settles
3	userId	ObjectId	Ref: User, Required	The user who made the payment
4	razorpayOrderId	String	Required	Razorpay-issued order ID (e.g., order_XXXXX)
5	razorpayPaymentId	String	Optional	Razorpay-issued payment ID — set on success

6	amount	Number	Required (in paise)	Amount charged — 5000 paise = Rs. 50
7	currency	String	Default: 'INR'	Transaction currency code
8	status	String	Enum: created/paid/failed	Payment transaction status
9	paidAt	Date	Optional	Timestamp of successful payment capture

3.7.5 Notifications Collection

Table 3.7: Notifications Collection Schema

Sr.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Primary Key, Auto	MongoDB auto-generated unique ID
2	recipientEmail	String	Required	Destination email address
3	subject	String	Required	Email subject line
4	bodyHtml	String	Required	Full HTML body of the email sent
5	type	String	Enum: booking_confirm/ payment_receipt/ schedule_change/ cancellation	Category of notification
6	sentAt	Date	Auto	Timestamp when email was dispatched by Nodemailer
7	relatedBookingId	ObjectId	Ref: Booking, Optional	Associated booking (if applicable)
8	isSuccessful	Boolean	Required	Whether Nodemailer reported successful dispatch

Chapter 4: Software Tools and Technologies

This chapter provides a thorough examination of each technology used in the development of Campus Cruiser. For each tool, the discussion covers its background, key technical characteristics, and specific role in the Campus Cruiser architecture. The selection criteria for each tool are also explained, enabling future maintainers and academic evaluators to understand the rationale behind the technology choices made by the project team — Akshay Anand, Arya Ratnam Sinha, and Abdul Fater.

4.1 Node.js

4.1.1 Background and History

Node.js was created by Ryan Dahl in 2009 and released as an open-source project. It was built on Google Chrome's V8 JavaScript engine, which compiles JavaScript directly to native machine code, providing exceptional performance. Prior to Node.js, server-side JavaScript was theoretically possible but impractical; Node.js established JavaScript as a first-class server-side language, fundamentally changing the web development landscape. Today, Node.js is maintained by the OpenJS Foundation and used by companies including Netflix, LinkedIn, Uber, PayPal, and NASA.

4.1.2 Core Technical Characteristics

Node.js's most distinctive characteristic is its event-driven, non-blocking I/O model. Unlike traditional synchronous server architectures (e.g., Apache with PHP), where each incoming request blocks a thread while waiting for database or file system operations, Node.js processes all I/O operations asynchronously. When a database query is issued, Node.js registers a callback and continues processing other requests. When the query result arrives, the event loop picks up the callback and processes the result. This model allows a single Node.js process to handle thousands of concurrent connections without the overhead of multi-threading.

The npm (Node Package Manager) ecosystem, bundled with Node.js, is the world's largest software registry with over 2 million packages. This rich ecosystem significantly accelerates

development by providing well-tested, production-ready solutions for virtually every backend need.

4.1.3 Role in Campus Cruiser

Node.js serves as the runtime environment for the entire Campus Cruiser backend. The Express.js API server, all middleware, route handlers, service classes (NotificationService, RazorpayService), and Mongoose database interactions all execute within the Node.js process. Node.js v18.x (LTS) was chosen for its long-term support status, ensuring security patches and maintenance.

4.2 Express.js

4.2.1 Overview

Express.js is the most widely used web application framework for Node.js, known for its minimalist, unopinionated philosophy. It provides a lightweight layer of fundamental web application features on top of Node's built-in HTTP module without obscuring Node.js features. Express applications are built by composing middleware functions and routing handlers.

4.2.2 Middleware Architecture

Express.js middleware is a function with the signature `(req, res, next) => {}`. The middleware pipeline in Campus Cruiser includes:

- `express.json()` — Parses incoming request bodies as JSON, making `req.body` available.
- `cors()` — Adds CORS headers allowing the React frontend to make API calls.
- `cookieParser()` — Parses Cookie headers for session-related functionality.
- `verifyFirebaseToken` — Custom middleware validating Firebase ID tokens on protected routes.
- `requireRole(role)` — Custom middleware enforcing role-based access control on admin routes.

- `errorHandler` — Global error handling middleware catching unhandled errors and returning structured JSON error responses.

4.2.3 Routing Structure in Campus Cruiser

Table 4.1: Express.js Routing Structure

Router File	Base Path	Key Endpoints
<code>authRouter.js</code>	<code>/api/auth</code>	POST <code>/register</code> , POST <code>/login</code> , GET <code>/me</code> , POST <code>/logout</code>
<code>scheduleRouter.js</code>	<code>/api/schedules</code>	GET <code>/</code> , POST <code>/ (admin)</code> , PUT <code>/:id (admin)</code> , DELETE <code>/:id (admin)</code>
<code>bookingRouter.js</code>	<code>/api/bookings</code>	POST <code>/create</code> , GET <code>/my</code> , GET <code>/:id</code> , POST <code>/:id/cancel</code>
<code>paymentRouter.js</code>	<code>/api/payments</code>	POST <code>/create-order</code> , POST <code>/webhook</code> , GET <code>/my</code>
<code>adminRouter.js</code>	<code>/api/admin</code>	GET <code>/stats</code> , GET <code>/users</code> , PUT <code>/users/:id</code> , GET <code>/bookings</code>

4.3 React.js

4.3.1 Overview and Component Model

React.js is an open-source JavaScript library for building user interfaces, developed by Meta (Facebook) and released in 2013. React introduced the concept of a component-based UI architecture and a virtual DOM (Document Object Model) for efficient rendering. A React application is composed of reusable components — independent JavaScript functions or classes that return JSX (JavaScript XML), a syntax extension allowing HTML-like code within JavaScript.

4.3.2 Key React Features Used in Campus Cruiser

- **Functional Components with Hooks:** All Campus Cruiser components are written as modern functional components. `useState` manages local component state; `useEffect` handles side effects (API calls on component mount); and `useContext` provides global state (authenticated user) without prop drilling.
- **React Router v6:** Provides client-side routing, enabling navigation between pages without full page reloads. Protected routes redirect unauthenticated users to `/login`.

- **Axios Integration:** Configured with a base URL and a request interceptor that automatically attaches the Firebase ID token from `localStorage` to every outbound request's Authorization header.
- **Controlled Forms:** All form inputs are controlled components, where React state is the single source of truth for input values, enabling real-time validation feedback.

4.3.3 Campus Cruiser Frontend Architecture

The React application is organised into the following directory structure: `/src/components/` (reusable UI components like `ScheduleCard`, `BookingModal`, `NavBar`), `/src/pages/` (full page components: `HomePage`, `SchedulesPage`, `BookingHistoryPage`, `AdminDashboard`), `/src/contexts/` (`AuthContext` providing Firebase user to all components), `/src/services/` (API service functions wrapping Axios calls), and `/src/utils/` (helper functions for date formatting, error handling, etc.).

4.4 MongoDB and Mongoose ODM

4.4.1 MongoDB — Document Database

MongoDB is a document-oriented NoSQL database developed by MongoDB Inc. and released in 2009. Data is stored in BSON (Binary JSON) documents — flexible, JSON-like structures that can contain nested objects and arrays without requiring a predefined schema. Collections (analogous to SQL tables) group related documents but impose no rigid schema, allowing documents in the same collection to have different structures.

MongoDB's flexibility makes it well-suited for an evolving application like Campus Cruiser, where data structures (e.g., booking attributes, schedule fields) may change between versions without requiring database migration scripts. MongoDB Atlas, the cloud-hosted version, was used for its free tier, automated backups, and ease of setup.

4.4.2 Mongoose ODM

Mongoose is an Object Document Mapper (ODM) library that adds a schema layer on top of MongoDB's flexibility. Mongoose schemas define the structure, data types, validators, and

default values for documents in a collection. While MongoDB itself would accept any document, Mongoose ensures that only well-formed documents that match the defined schema are saved.

Key Mongoose features used in Campus Cruiser include: schema validation (enforcing required fields, enum constraints, min/max values), virtual populate (linking related documents across collections), middleware hooks (pre-save hooks for bookingRef generation), and timestamps (automatic createdAt/updatedAt fields on all documents).

4.5 Firebase Authentication

4.5.1 Overview

Firebase Authentication is a backend-as-a-service identity management platform by Google. It supports email/password, phone, and over 15 social sign-in providers. Campus Cruiser uses the email/password provider exclusively, with domain restriction enforced at the application level by validating that registration emails end with @cgc.edu.in before calling Firebase.

4.5.2 Token-Based Authentication Flow

Firebase Authentication uses JWTs (JSON Web Tokens) as the session mechanism. When a user logs in, Firebase issues a short-lived ID token (1-hour expiry) and a refresh token (valid until revoked). The Campus Cruiser backend's auth middleware verifies this token on every protected request using the Firebase Admin SDK's `verifyIdToken()` method, which validates the token's signature against Google's public key certificates.

4.5.3 Security Advantages

- **Password Security:** Firebase handles password hashing, salting, and storage using enterprise-grade cryptographic standards. The application backend never receives or stores raw passwords.
- **Email Verification:** Firebase automatically sends a verification email upon registration. The backend checks `emailVerified` in the decoded token, preventing unverified accounts from making bookings.

- **Token Revocation:** When an admin suspends a user account, the backend calls Firebase Admin SDK's `revokeRefreshTokens(uid)` to invalidate all active sessions for that user.
- **Brute Force Protection:** Firebase Authentication has built-in rate limiting and CAPTCHA challenges for repeated failed login attempts.

4.6 Razorpay Payment Gateway

4.6.1 About Razorpay

Razorpay is one of India's leading payment technology companies, providing a full-stack payments and financial infrastructure platform. Founded in 2014, Razorpay processes billions of transactions annually for hundreds of thousands of Indian businesses. Its developer-first philosophy — characterised by comprehensive API documentation, an excellent sandbox environment, and a well-maintained Node.js SDK — makes it the preferred choice for developer-led projects in India.

4.6.2 Supported Payment Methods

Table 4.2: Razorpay Supported Payment Methods

Payment Method	Details	Support in Campus Cruiser
UPI	PhonePe, Google Pay, BHIM UPI, Paytm UPI	Yes
Net Banking	All major Indian banks (SBI, HDFC, ICICI, Axis, etc.)	Yes
Credit Cards	Visa, Mastercard, American Express, RuPay	Yes
Debit Cards	All major Indian bank debit cards	Yes
Wallets	Paytm, Mobikwik, Freecharge, Airtel Money	Yes
EMI	Credit card EMI options	Not implemented (not needed for Rs. 50)

4.6.3 Webhook Security

Razorpay's webhook system provides a secure mechanism for backend payment confirmation that does not rely on the frontend (which could be manipulated by a malicious user). When a payment is captured, Razorpay sends a POST request to the configured webhook URL with a

JSON payload and an X-Razorpay-Signature header. The signature is computed as HMAC-SHA256(rawPayload, webhookSecret). The backend independently computes the same HMAC and compares it with the provided signature. If they match, the payment is authentic. This prevents any attempt to confirm a booking by sending a forged webhook event.

4.7 Nodemailer

4.7.1 Overview

Nodemailer is the most popular Node.js library for sending emails. It supports SMTP, sendmail, SES (Amazon Simple Email Service), and other transport methods. Campus Cruiser configures Nodemailer with Gmail's SMTP server using an app-specific password generated from the institute's Gmail account, avoiding the need to expose the primary account password.

4.7.2 Email Template Design

All Campus Cruiser emails use inline HTML/CSS templates to ensure compatibility across email clients (Gmail, Outlook, Apple Mail). Each template includes the Campus Cruiser logo (as an inline base64 image), a header banner in the institute's colour scheme, the email body content, and a footer with the institute name and unsubscribe instructions. The booking confirmation template includes a table summarising booking details (route, date, time, seat number, booking reference, amount paid) formatted for quick readability.

4.8 Additional Libraries and Tools

Table 4.3: Additional Libraries and Tools

Library	Version	Role in Campus Cruiser	Why Selected
bcryptjs	v2.4.3	Hashing locally-managed passwords	Pure-JS bcrypt; no native bindings required
jsonwebtoken	v9.0.2	Custom JWT generation for internal service tokens	Industry-standard; supports HS256/RS256
cookie-parser	v1.4.6	Parsing Cookie headers in Express middleware	Standard Express companion; minimal overhead

cors	v2.8.5	CORS header injection for cross-origin requests	Simple, configurable; supports whitelist pattern
dotenv	v16.4.5	Loading environment variables from .env files	Essential for secret management
multer	v1.4.5	Multipart form data parsing for file uploads	Disk/memory storage options; integrates cleanly with Express
axios	v1.6.8	HTTP client for frontend API calls	Promise-based; interceptor support for auth token
mongoose	v8.2.3	MongoDB ODM for schema management and queries	De facto standard for Node.js MongoDB applications
crypto	v1.0.1	HMAC-SHA256 for Razorpay webhook signature verification	Built into Node.js stdlib; no extra dependency
nodemon	Dev only	Auto server restart on file changes during development	Improves development velocity

Chapter 5: Implementation and Testing

5.1 Development Methodology

5.1.1 Agile Scrum Framework

Campus Cruiser was developed using the Agile Scrum framework, adapted for a three-person team. The project was divided into 7 two-to-three-week sprints. Each sprint began with a planning session where the team reviewed the product backlog, estimated story points for user stories, and committed to a sprint goal. Each sprint ended with a sprint review (demo of working features) and a retrospective (process improvement discussion).

The team used GitHub as the primary collaboration platform. Each feature was developed on a dedicated branch following the naming convention `feature/<feature-name>` (e.g., `feature/razorpay-integration`). Pull requests required at least one peer review before merging to the main branch. GitHub Issues tracked the product backlog, with labels for enhancement, bug, and documentation.

5.1.2 Team Responsibilities

Table 5.1: Team Responsibilities

Team Member	Primary Responsibilities	Secondary Contributions
Akshay Anand	Backend architecture, Express.js API setup, Firebase Auth middleware, Admin module backend	Code reviews, deployment setup, API documentation
Aarya Ratnam Sinha	React.js frontend, all UI components, responsive design, React Router setup, Axios integration	Frontend testing, UAT coordination, project report writing
Abdul Fater	Booking module, Razorpay integration, webhook handling, Nodemailer notification service	Integration testing, security testing, database schema design

5.1.3 Version Control and Branching Strategy

The team followed a feature-branch workflow. The main branch always contained a deployable version of the code. Development branches were merged via pull requests after peer review. Hotfixes were made on hotfix/ branches and merged directly to main. The .env file was included in .gitignore and shared securely between team members via an encrypted team vault, not through the repository.

5.2 Backend Implementation Details

5.2.1 Project Structure

The backend (in the /api directory) is organised as follows: /api/index.js (Express app entry point), /api/routes/ (router modules for each feature domain), /api/models/ (Mongoose schema models), /api/middleware/ (auth and role middleware), /api/services/ (NotificationService, RazorpayService), /api/controllers/ (business logic handlers called by routes), and /api/utils/ (helper functions: bookingRefGenerator, emailTemplates).

5.2.2 Authentication Middleware Implementation

The verifyFirebaseToken middleware extracts the Authorization header, validates its 'Bearer <token>' format, and passes the token to Firebase Admin SDK's auth().verifyIdToken(token). This method performs the following validation steps: (1) Checks the token signature against Google's current public key certificates. (2) Verifies that the token has not expired (tokens are valid for 1 hour). (3) Verifies the iss (issuer) and aud (audience) claims match the Firebase project. (4) Checks that the token has not been revoked. On success, the decoded token object (containing uid, email, emailVerified) is attached to req.user and next() is called. On failure, a 401 response is returned immediately.

5.2.3 Atomic Seat Booking Implementation

The seat booking operation is implemented using MongoDB's findOneAndUpdate with a condition-based update to ensure atomicity. The operation atomically checks the condition (availableSeats > 0) and performs the decrement: findOneAndUpdate({ _id: scheduleId, availableSeats: { \$gt: 0 } }, { \$inc: { availableSeats: -1 } }, { new: true }). MongoDB's document-level locking ensures that two simultaneous requests cannot both satisfy the condition and

decrement the count below zero. This is significantly more reliable than a read-then-write approach, which would be vulnerable to race conditions.

5.2.4 Razorpay Webhook Implementation

The webhook endpoint `/api/payments/webhook` accepts raw request bodies (using `express.raw()` middleware, not `express.json()`, to preserve the exact bytes for signature verification). The handler: (1) Extracts the `X-Razorpay-Signature` header. (2) Computes HMAC-SHA256 of the raw body string using the `RAZORPAY_WEBHOOK_SECRET` environment variable. (3) Compares the computed hash with the provided signature using `crypto.timingSafeEqual()` to prevent timing attacks. (4) If valid and the event type is `'payment.captured'`, updates `booking.status` to `'confirmed'` and creates a `Payment` document. (5) Triggers `NotificationService.sendBookingConfirmation()`.

5.3 Frontend Implementation Details

5.3.1 Authentication Context

The `AuthContext` (in `/src/contexts/AuthContext.js`) wraps the entire application and provides the current Firebase user object to all components via React Context. The context subscribes to Firebase's `onAuthStateChanged()` listener, which fires whenever the user's authentication state changes (login, logout, token refresh). The current user object and loading state are provided via `useContext(AuthContext)` to any component that needs authentication information.

5.3.2 Protected Route Implementation

The `PrivateRoute` component wraps any route that requires authentication. It checks the current user from `AuthContext`. If loading (waiting for Firebase to restore session), it renders a loading spinner. If the user is null (not authenticated), it redirects to `/login` using React Router's `Navigate` component. If the user is authenticated, it renders the wrapped component. `AdminRoute` extends `PrivateRoute` with an additional role check, redirecting non-admin users to the home page with an 'Access Denied' toast notification.

5.3.3 Razorpay Frontend Integration

The Razorpay checkout widget is loaded by dynamically injecting the Razorpay script (<https://checkout.razorpay.com/v1/checkout.js>) into the page. The PaymentModal component creates a Razorpay instance with the order ID, prefill data, and handler callbacks. The handler.payment_success callback sends the payment ID, order ID, and signature to the backend for server-side verification. The handler.payment_failed callback restores the UI to the pre-payment state and displays an error message.

5.4 User Interface Descriptions

5.4.1 Home Page (/)

The home page serves as the public landing page for Campus Cruiser. It features a full-width hero section with the tagline 'Book Your Campus Bus — Anytime, Anywhere', a call-to-action button ('Get Started'), and a brief animated feature list highlighting the platform's three core benefits: Advance Booking, Secure Payments, and Instant Confirmations. The navigation bar displays links to the Login and Register pages for unauthenticated users, and the Dashboard link for authenticated users. The footer contains the institute logo, contact information, and a link to the transportation office.

5.4.2 Registration Page (/register)

The registration page presents a two-column card layout. The left column contains an informational panel explaining Campus Cruiser's benefits and access requirements. The right column contains the registration form with fields for Full Name, Email (with real-time domain validation — showing a red error indicator for non-@cgc.edu.in addresses), Role (Student/Faculty radio buttons), Phone Number (optional), and Password (with strength indicator). The Submit button is disabled until all required fields pass validation. On successful registration, a toast notification instructs the user to check their email for the verification link.

5.4.3 Schedule Browsing Page (/schedules)

The schedule listing page is accessible to all authenticated users. At the top, a filter bar allows searching by destination (text input with autocomplete), filtering by time range

(morning/afternoon/evening), and selecting a date. Below the filter bar, active schedules are displayed in a responsive card grid (3 columns on desktop, 2 on tablet, 1 on mobile). Each card shows the route name, source → destination with a bus icon, departure and arrival times, remaining seats (with a colour-coded badge: green for >10, amber for 5–10, red for <5), fare, and a 'Book Now' button. Fully booked schedules show a grey 'Fully Booked' badge instead.

5.4.4 Booking Modal

When a user clicks 'Book Now', a modal overlay appears with the full trip details and a trip date selector (a date picker restricted to future dates). Below the date picker, the total fare is displayed. For Student users, a 'Proceed to Payment' button opens the Razorpay checkout widget. For Faculty users, a toggle switch allows switching between 'Pay Now' and 'Pay Later (Deferred)'. A summary section at the bottom of the modal shows the user's current outstanding balance if they are a Faculty member with deferred dues.

5.4.5 Booking History Page (/my-bookings)

The booking history page displays a paginated list of the authenticated user's bookings, sorted by trip date (upcoming first). Each entry in the list shows: booking reference, route name, trip date and time, booking status (with colour-coded badges: green for confirmed, amber for pending, red for cancelled), payment mode and status, and action buttons (View Details, Cancel — only shown for future confirmed bookings). Clicking 'View Details' expands an accordion with the full booking information including the Razorpay payment ID.

5.4.6 Admin Dashboard (/admin)

The admin dashboard uses a sidebar navigation layout with the following sections:

- Overview: Summary metric cards (Today's Bookings, Today's Revenue, Total Active Schedules, Total Users) with trend indicators comparing to yesterday's figures.
- Schedules: A full CRUD data table with inline edit (opens a right-side drawer form), delete (with active booking check), and 'Add Schedule' button. Supports sorting by route name, departure time, and available seats.

- **Bookings:** A filterable, exportable data table of all bookings. Filters: status, date range, route, user email. Actions: View Details, Cancel Booking (with confirmation dialog and auto-notification).
- **Users:** A searchable table of all registered users with columns for name, email, role, registration date, active status, and outstanding balance. Actions: Toggle Active/Suspended, View Bookings.
- **Payments:** A transaction ledger with columns: booking reference, user, amount, Razorpay IDs, status, and timestamp. Deferred balances are highlighted. A 'Send Payment Reminder' button is available for accounts with outstanding dues.

5.5 Testing Methodology

5.5.1 Unit Testing

Unit testing focused on individual functions and components in isolation. Backend utility functions were the primary unit test targets:

- `generateBookingRef()`: Tested with 50 sequential invocations to verify unique output, correct prefix format (CC-YYYY-NNNNN), and no collisions.
- `verifyRazorpaySignature(body, signature, secret)`: Tested with known test vectors from Razorpay's documentation to confirm correct HMAC-SHA256 computation.
- `emailTemplates.bookingConfirmation(data)`: Tested by rendering the template with mock data and asserting that required fields (booking reference, route, amount) appear in the output HTML.

Frontend component unit tests were written using React Testing Library (RTL) and Jest. The `ScheduleCard` component was tested to verify correct rendering of available/unavailable states. The `BookingModal` component was tested to verify form validation behaviour and payment flow branching (Student vs. Faculty).

5.5.2 Integration Testing

Integration tests verified the correct interaction between connected modules. The booking-to-payment integration test flow covered: (1) creating a test user account, (2) creating a test schedule with 2 seats, (3) making a booking (POST /api/bookings/create), (4) verifying the provisional booking in MongoDB, (5) simulating a Razorpay webhook with a valid signature, (6) verifying booking status updated to 'confirmed', (7) verifying a notification record was created in the Notifications collection.

The auth middleware integration was tested by making requests to protected endpoints without a token (expecting 401), with an expired token (expecting 401), with a valid student token to an admin endpoint (expecting 403), and with a valid admin token to an admin endpoint (expecting 200). All four tests passed on the first integration test run.

5.5.3 System Testing

System testing involved end-to-end testing of complete user journeys on the deployed application. Three full user journey scenarios were executed:

33. Student Journey: Register (with @cgc.edu.in email) → Verify email → Login → Browse schedules → Book trip → Complete Razorpay payment (test mode) → Receive confirmation email → View booking in history → Cancel booking → Verify seat count restored.
34. Faculty Journey: Login as faculty → Browse schedules → Book trip → Select 'Pay Later' → Verify booking confirmed → Verify outstanding balance updated → Admin views deferred balance → Admin sends payment reminder email.
35. Admin Journey: Login as admin → View dashboard metrics → Add new schedule → Verify schedule appears in public listing → Modify schedule departure time → Verify affected users received email notification → View all bookings → Cancel a confirmed booking → Verify user received cancellation email and seat count restored.

All three journeys completed successfully without errors in the system testing phase.

5.5.4 Security Testing

Security testing focused on the authentication, authorisation, and payment security layers:

- Token Bypass Attempt: Directly accessing `/api/bookings/create` without Authorization header → 401. Correct behaviour confirmed.
- Role Escalation Attempt: Making a DELETE `/api/admin/schedules/:id` request with a valid student token → 403. Confirmed.
- Overbooking Injection: Simultaneously firing 10 concurrent booking requests for a schedule with 1 available seat → Exactly 1 booking succeeded; 9 returned 409. Atomic MongoDB update confirmed effective.
- Forged Webhook: Sending a payment.captured webhook with a fabricated signature → 400 Bad Request; booking status unchanged. HMAC verification confirmed effective.
- NoSQL Injection: Submitting `{sne: null}` as the scheduleId in a booking request → Mongoose schema validation rejected the non-ObjectId value before the database was queried. Injection blocked.

5.5.5 User Acceptance Testing (UAT)

User Acceptance Testing was conducted with 15 participants drawn from the Chandigarh Engineering College-CGC Landran student and faculty community. Participants were given a test account, a brief onboarding document, and a set of tasks to complete (register, browse, book, pay, view history). The UAT coordinator — Aarya Ratnam Sinha — observed sessions and collected structured feedback using a task completion form.

Table 5.2: UAT Results Summary

UAT Metric	Result
Registration task completion rate	100% (15/15 participants)
Schedule browsing task completion rate	100% (15/15 participants)
Booking task completion rate	93% (14/15 — one participant confused by date picker; resolved post-UAT)
Payment task completion rate (Razorpay test mode)	100% (15/15 participants)
Average registration time	1 minute 48 seconds
Average booking time (from schedule page)	1 minute 22 seconds
Overall satisfaction score (1–5)	4.4 / 5.0
Would recommend to other students	93% (14/15 participants)

5.6 Test Cases and Results

Table 5.3: Test Cases and Results

TC ID	Module	Test Scenario	Expected Output	Actual Output	Status
TC01	Auth	Register with valid @cgc.edu.in email	Account created; verification email sent	As expected	Pass
TC02	Auth	Register with @gmail.com email	Frontend validation blocks submission; error shown	As expected	Pass
TC03	Auth	Login with correct credentials	Firebase token issued; user redirected to dashboard	As expected	Pass
TC04	Auth	Login with incorrect password	Firebase error; 'Invalid credentials' message displayed	As expected	Pass
TC05	Auth	Login with unverified email	Login blocked; 'Verify your email first' message shown	As expected	Pass
TC06	Auth	Access protected API without token	401 Unauthorized response from backend	As expected	Pass
TC07	Auth	Access admin API with student token	403 Forbidden; admin privileges required	As expected	Pass
TC08	Auth	Token expiry auto-logout	Firebase auto-refresh fails; user redirected to login	As expected	Pass
TC09	Schedule	Admin creates new schedule	Schedule saved; appears in public schedule listing	As expected	Pass
TC10	Schedule	Admin edits departure time	Schedule updated; affected bookings receive email alert	As expected	Pass
TC11	Schedule	Admin deletes schedule with no bookings	Schedule deleted; no longer visible in public listing	As expected	Pass
TC12	Schedule	Admin deletes schedule with active bookings	Warning shown; deactivation offered instead of deletion	As expected	Pass
TC13	Booking	Student books available seat	Provisional booking created; availableSeats decremented	As expected	Pass
TC14	Booking	Booking fully-booked schedule	409 Conflict; 'No seats available' error message	As expected	Pass
TC15	Booking	10 concurrent booking requests for 1 seat	Exactly 1 success; 9 receive 409 Conflict	As expected	Pass
TC16	Booking	Student cancels future booking	Booking cancelled; availableSeats restored; cancellation email sent	As expected	Pass
TC17	Payment	Student completes Razorpay payment	Payment captured; booking status confirmed; email sent	As expected	Pass

TC18	Payment	Razorpay payment fails (declined card)	Payment failed; booking remains pending; error shown to user	As expected	Pass
TC19	Payment	Forged webhook with invalid signature	400 Bad Request; signature mismatch; booking unchanged	As expected	Pass
TC20	Payment	Faculty selects deferred payment	Booking confirmed immediately; outstandingBalance += 50	As expected	Pass
TC21	Notification	Booking confirmation email dispatched	HTML email received with correct booking details	As expected	Pass
TC22	Notification	Schedule change email to affected users	All users with confirmed bookings receive alert email	As expected	Pass
TC23	Notification	Cancellation confirmation email	User receives cancellation email with refund timeline	As expected	Pass
TC24	Admin	Admin dashboard metrics accuracy	Bookings and revenue counts match MongoDB aggregation	As expected	Pass
TC25	UI	Responsive layout on mobile (375px)	All pages render without horizontal scroll or overflow	As expected	Pass

Chapter 6: Conclusion and Future Work

6.1 Conclusion

Campus Cruiser stands as a successful demonstration of how a focused, well-engineered web application can meaningfully improve the quality of daily life in an academic institution. Over the course of fifteen weeks, Akshay Anand, Aarya Ratnam Sinha, and Abdul Fater designed, developed, tested, and deployed a fully functional campus transportation management system for Chandigarh Engineering College-CGC Landran — replacing a fragmented, manual workflow with a seamless digital platform.

The project achieved all eight of its stated objectives. Secure, domain-restricted access was implemented through Firebase Authentication with @cgc.edu.in email verification, ensuring the platform is used exclusively by the Chandigarh Engineering College-CGC Landran community. Real-time schedule browsing with filtering and live seat availability display was delivered through a responsive React.js interface backed by a MongoDB data store. Atomic concurrency-safe seat booking using MongoDB's `findOneAndUpdate` operation successfully prevents overbooking even under simultaneous high-traffic conditions.

Razorpay payment integration supporting UPI, net banking, and card payments was implemented with HMAC-SHA256 webhook verification, providing a secure and reliable online payment experience. A faculty-specific deferred payment option with outstanding balance tracking was implemented — a feature absent from all reviewed prior work, directly addressing a key user need. A comprehensive admin dashboard providing full control over schedules, bookings, users, and payments was delivered, requiring no technical knowledge to operate.

Automated email notifications via Nodemailer for booking confirmations, payment receipts, schedule changes, and cancellations were implemented with professionally designed HTML email templates. Near-zero infrastructure cost was achieved through strategic use of free-tier services (MongoDB Atlas, Firebase, Razorpay sandbox, Vercel), making the system economically viable for long-term institutional deployment.

From a technical standpoint, the project demonstrates the power and productivity of the JavaScript full-stack paradigm. The MERN-adjacent stack (Node.js, Express.js, React.js, MongoDB) enabled rapid iteration, clear code organisation, and seamless data flow between layers. The integration of Firebase Authentication, Razorpay, and Nodemailer demonstrates how

modern web applications can leverage best-in-class third-party services to deliver enterprise-grade features without proportional development cost.

Beyond its technical merits, Campus Cruiser has provided the development team with deep, practical experience that complements and extends the theoretical knowledge gained through coursework — in full-stack architecture design, RESTful API development, database modelling, payment gateway integration, security engineering, and Agile project management. These competencies represent a strong foundation for careers in software engineering and technology entrepreneurship.

6.2 Limitations

The current version of Campus Cruiser, while fully functional, has several limitations that have been identified for future resolution:

- **Absence of Real-Time GPS Tracking:** The system provides static schedule information but cannot show users the live location of a bus or provide dynamic ETA updates in case of delays or traffic.
- **Web-Only Platform:** Campus Cruiser is a responsive web application and does not have a dedicated native mobile application. While the web app functions on mobile browsers, it lacks native mobile capabilities such as offline access and push notifications.
- **No ERP or SIS Integration:** User registration is entirely self-service. There is no automatic synchronisation with Chandigarh Engineering College-CGC Landran's student enrolment database.
- **Limited Reporting and Analytics:** The admin dashboard provides basic operational metrics but lacks advanced analytics — no trend charts, demand forecasting, route utilisation heatmaps, or exportable reports.
- **Automatic Seat Assignment Only:** Users cannot select a specific seat on a bus. All seat assignments are implicit (booking = any seat).
- **Single Currency and Region:** The payment system is configured only for INR and Indian payment methods.

6.3 Future Enhancements

6.3.1 Progressive Web Application (PWA) and Push Notifications

Converting Campus Cruiser into a Progressive Web Application would bring it significantly closer to a native mobile app experience without requiring separate iOS/Android codebases. A PWA with a service worker would enable: (a) offline access to previously loaded schedules via cached data; (b) 'Add to Home Screen' installation on mobile devices; (c) push notifications via the Web Push Protocol — critical for delivering time-sensitive alerts. Push notifications would replace reliance on email for time-critical communications.

6.3.2 Real-Time GPS Bus Tracking

The most impactful enhancement would be the integration of real-time GPS tracking for campus buses. This would require: (a) installation of low-cost GPS tracker devices on each campus bus; (b) a backend WebSocket server (using Socket.io) that receives GPS position updates from the trackers; (c) a map interface on the frontend (using Google Maps API or the open-source Leaflet.js library) showing live bus positions, routes, and ETAs. This feature alone would dramatically improve the usefulness of Campus Cruiser during actual commutes.

6.3.3 AI-Based Demand Forecasting and Schedule Optimisation

As Campus Cruiser accumulates historical booking data over multiple semesters, machine learning models trained on this data can provide significant value. Proposed applications include: (a) predicting peak demand periods (examination dates, festival days, event days) to proactively schedule extra trips; (b) identifying chronically under-utilised routes that could be consolidated to reduce operational costs; (c) recommending optimal departure times based on historical booking patterns.

6.3.4 QR Code Digital Ticketing

Each confirmed booking would generate a unique QR code ticket, displayable on the user's phone from the booking history page. Bus conductors would use a simple mobile scanning application to scan and validate QR codes at boarding. This would eliminate the need for any paper ticketing

or cash exchange at the point of boarding, making the boarding process faster, more hygienic, and fully auditable.

6.3.5 ERP and SIS Integration

Integration with Chandigarh Engineering College-CGC Landran's existing Student Information System (SIS) would allow: (a) automatic account provisioning for new students upon enrolment; (b) automatic account deactivation upon graduation or departure; (c) access to enrolment data (department, year, programme) for enhanced reporting; (d) single sign-on (SSO) using the institute's existing LDAP/Active Directory infrastructure.

6.3.6 Seat Selection Interface

A visual seat map for each scheduled bus would allow users to choose their preferred seat (e.g., window vs. aisle, front vs. rear) at the time of booking. The seat map would be generated dynamically based on the bus layout stored in the Schedule document, and selected seats would be locked in real time as users browse the map — similar to the flight or cinema seat booking experience.

6.3.7 Multi-Tenant Architecture for Multi-Campus Deployment

Refactoring Campus Cruiser into a multi-tenant architecture would allow the platform to serve multiple campuses or other academic institutions from a single backend deployment. Each tenant (institution) would have isolated data (separate MongoDB namespaces), configurable branding (logo, colour scheme, domain restriction), and independent admin access. This architectural evolution would transform Campus Cruiser from an institutional tool into a scalable commercial or open-source platform product.

In summary, Campus Cruiser in its current form is a production-ready MVP that delivers immediate, measurable value to the Chandigarh Engineering College-CGC Landran transportation ecosystem. With the enhancements described above, it has the potential to evolve into a comprehensive, AI-augmented, multi-institution campus mobility platform — a

meaningful contribution to the broader challenge of sustainable, smart campus infrastructure in Indian higher education.

References

Books

- Flanagan, D. (2020). JavaScript: The Definitive Guide (7th Edition). O'Reilly Media.
- Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Apps (2nd Edition). O'Reilly Media.
- Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming (3rd Edition). No Starch Press.
- Chodorow, K. (2013). MongoDB: The Definitive Guide — Powerful and Scalable Data Storage (2nd Edition). O'Reilly Media.
- Lauret, A. (2019). The Design of Web APIs. Manning Publications.
- Freeman, E. (2020). Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software (2nd Edition). O'Reilly Media.

Research Papers and Journals

- Priya, S., Venkataraman, R., & Krishnamurthy, P. (2019). A Web-Based Campus Transportation Booking System for Engineering Colleges. International Journal of Engineering and Technology, 8(3), 112–118.
- Sharma, R., & Gupta, A. (2021). Design and Development of Campus Shuttle Management System Using MERN Stack. Journal of Computer Science Applications, 12(4), 78–85.
- Mehta, V., & Choudhary, S. (2022). React Native-Based Campus Transport Booking Application with Push Notification Support. Proceedings of the 14th International Conference on Mobile Systems and Applications, 312–318.
- Kulkarni, N., Deshmukh, P., & Patil, V. (2022). Comparative Evaluation of Payment Gateways for Indian Academic Platforms: A Developer-Centric Study. Proceedings of the National Conference on Emerging Technologies in Computing, 215–220.

- Patel, R., & Rao, S. (2020). Firebase Authentication for Institutional Web Platforms: Implementation Patterns and Security Analysis. *International Journal of Information Systems and Computer Science*, 9(1), 55–63.
- Galanaki, E. (2002). The Decision to Recruit Online: A Descriptive Study. *Career Development International*, 7(4), 243–251.
- Misra, S., Singh, R., & Agarwal, A. (2021). Cloud-Native Web Application Development Using Serverless and BaaS Platforms: A Case Study. *IEEE Access*, 9, 142711–142724.

Official Documentation

- Node.js Foundation. (2024). Node.js v18.x Documentation. Retrieved from <https://nodejs.org/en/docs/>
- OpenJS Foundation. (2024). Express.js v4.x API Reference. Retrieved from <https://expressjs.com/en/4x/api.html>
- Meta Platforms. (2024). React.js Official Documentation. Retrieved from <https://react.dev/>
- MongoDB, Inc. (2024). MongoDB Manual v6.0. Retrieved from <https://www.mongodb.com/docs/manual/>
- Mongoose Contributors. (2024). Mongoose ODM v8.x Documentation. Retrieved from <https://mongoosejs.com/docs/>
- Google Firebase. (2024). Firebase Authentication Documentation. Retrieved from <https://firebase.google.com/docs/auth>
- Razorpay, Inc. (2024). Razorpay Developer Documentation — Node.js Integration. Retrieved from <https://razorpay.com/docs/api/>
- Andris Reinman. (2024). Nodemailer Documentation. Retrieved from <https://nodemailer.com/about/>
- Axios Contributors. (2024). Axios v1.x Documentation. Retrieved from <https://axios-http.com/docs/intro>

GitHub Repository

- Anand, A., Sinha, A. R., & Fater, A. (2024). Campus Cruiser: Smart Campus Transportation Management System. GitHub Repository. Retrieved from https://github.com/sssangamkrmishra-dev/campus_cruiser